

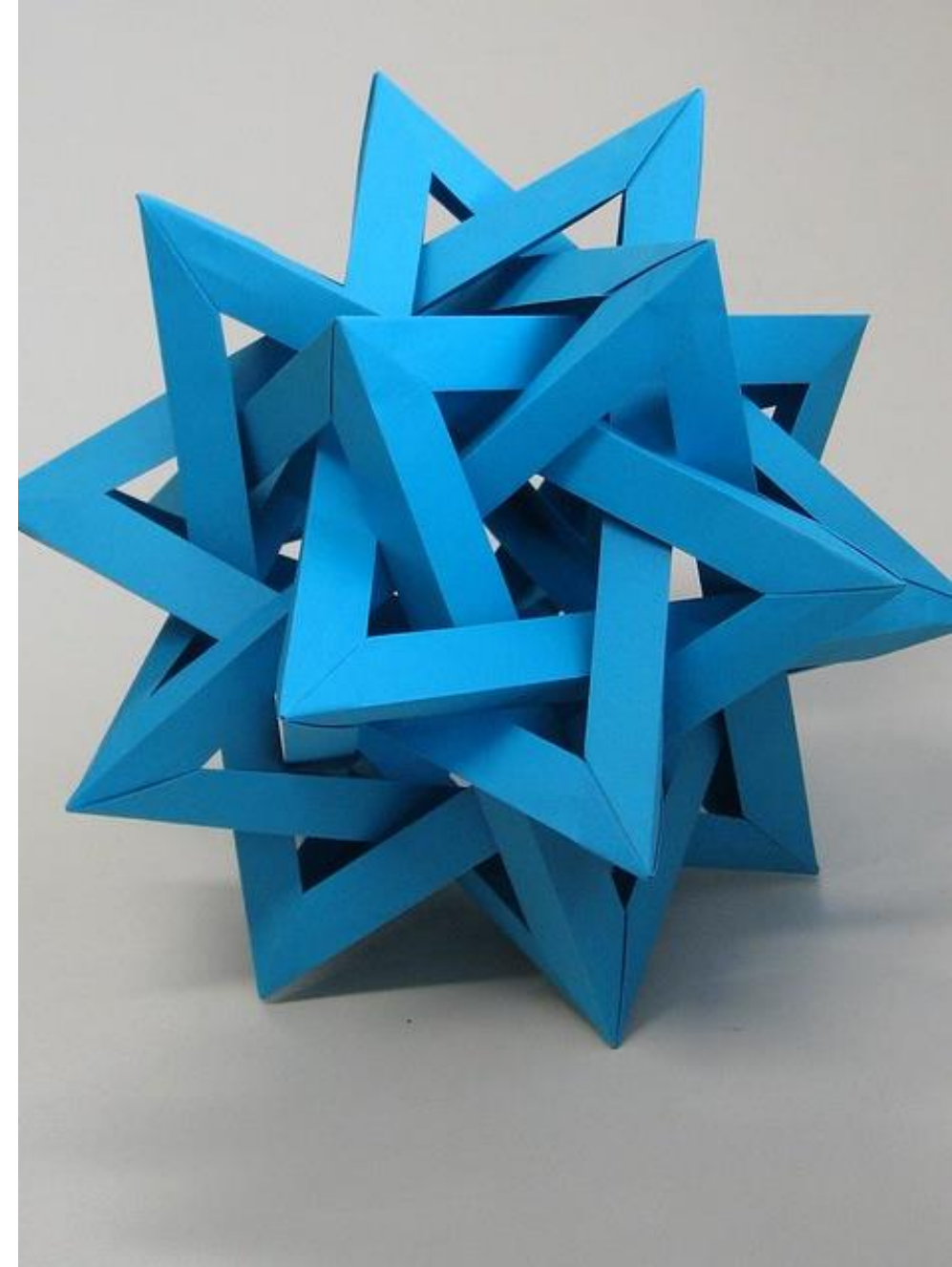


Unit P6: Lists and Tables

LISTS, OPERATION ON LISTS, NESTED LISTS



Chapter 6



Unit Goals

- To collect elements using lists
- To use the for loop for traversing lists
- To learn common algorithms for processing lists
- To use lists with functions
- To work with tables of data

Contents

- Basic Properties of Lists
- List Operations
- Common List Algorithms
- Using Lists with Functions
- Problem Solving: Adapting Algorithms
- Tables

What is a List?

- A List is a versatile dynamic **data structure**, that stores a variable number of elements, of **any type**, that may be accessed by position (index)
 - Other programming languages call it: Sequence, Array, Vector, etc.
- The only other “container” of elements we have seen before is a **string**, but with some limitations:
 - Elements can only be **characters**
 - **Immutable**

Basic Properties of Lists



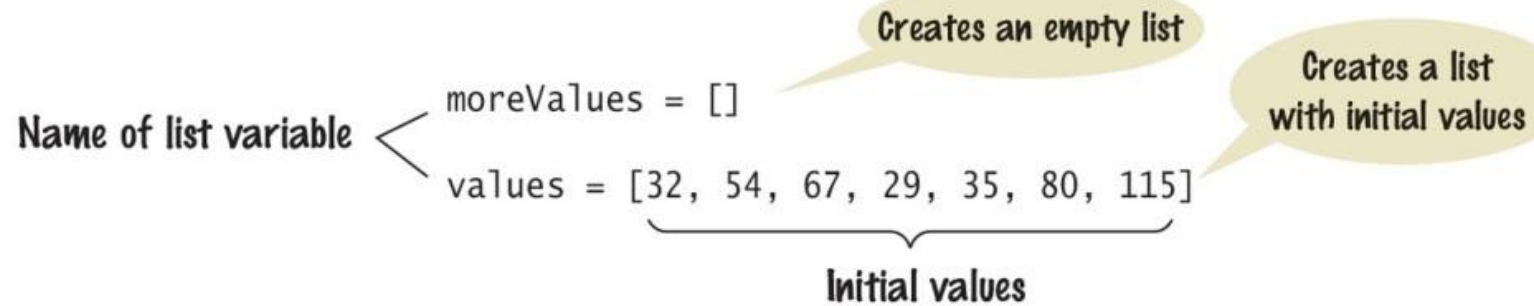
6.1

Creating a List

- Specify a list variable with the subscript operator `[]`

Syntax

To create a list:	<code>[value₁, value₂, . . .]</code>
To access an element:	<code>listReference[index]</code>

Name of list variable  `moreValues = []` Creates an empty list
`values = [32, 54, 67, 29, 35, 80, 115]` Creates a list with initial values
Initial values

Use brackets to access an element.

`values[i] = 0`
`element = values[i]`

Accessing List Elements

- A list is a sequence of elements, each of which has an integer **position** or **index**
- To access a list **element**, you specify which **index** you want to use. That is done with the subscript operator (in the same way that you access individual characters in a string)
- Indexes start at 0

Accessing list
elements

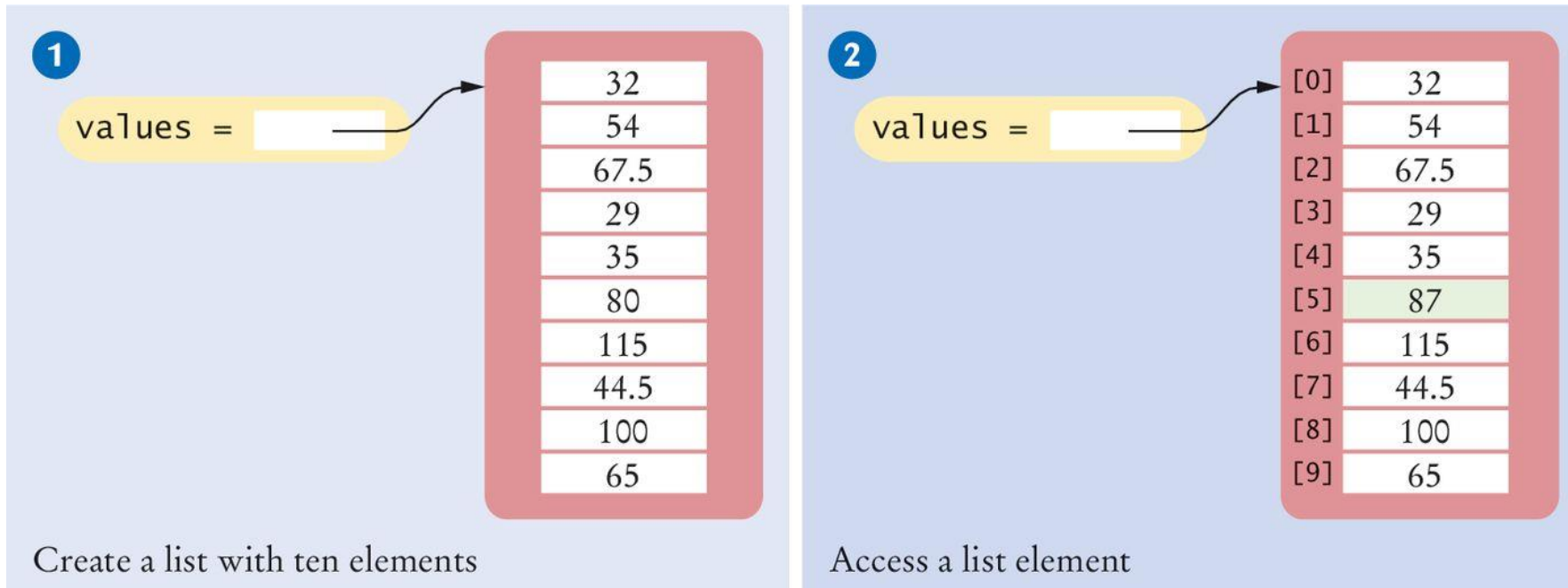
```
print(values[5])
```

Replacing list
elements

```
values[5] = 87
```

**Different from strings: we
can replace an element**

Creating Lists/Accessing Elements



```
# 1: Creating a list  
values = [32, 54, 67.5, 29, 35, 80, 115, 44.5, 100, 65]
```

```
# 2: Accessing a list element  
values[5] = 87
```


Lists vs. Strings

- Both lists and strings are **sequences**, and the `[]` operator is used to access an element in any sequence
- There are two differences between lists and strings:
 - **Lists** can hold values of **any type**, whereas **strings** are sequences of **characters**
 - Moreover:
 - Strings are immutable – you cannot change the characters in the sequence
 - Lists are mutable – the values of an element may be updated, new elements may be added, elements may be deleted

Types of Elements in a List

- A list of integer values

```
short_months = [ 2, 4, 6, 9, 11 ]
```

- A list of real values

```
math_constants = [ 3.1415, 2.718 ]
```

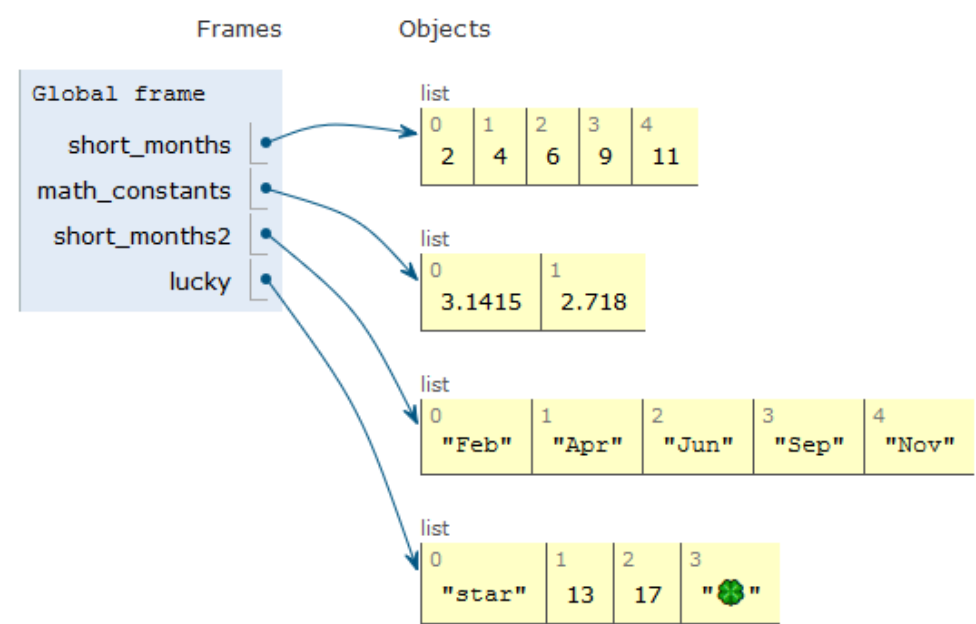
- A list of strings

```
short_months2 = [ 'Feb', 'Apr', 'Jun', 'Sep', 'Nov' ]
```

- A list with **mixed values**

- lucky = ['star', 13, 17, '🍀']

- **Unless you have a good reason, avoid mixing types**



Out of Range Errors

- **Out-of-Range** Errors:
- As in the case of strings, perhaps the most common error in using lists is **accessing a nonexistent element**

```
values = [2.3, 4.5, 7.2, 1.0, 12.2, 9.0, 15.2, 0.5]
values[8] = 5.4
# Error -- values has 8 elements,
# and the index can range from 0 to 7
```

- If your program accesses a list through an out-of-range index, the program will generate an **exception** at run time

Printing a list

- A list may be used with the `print()` function
- All the elements are printed, with a format resembling the one used for list creation

```
>>> values = [ 1, 2, 3 ]  
>>> print(values)  
[1, 2, 3]  
>>> print(values[0])  
1
```

Determining the List Length

- You can use the `len()` function to obtain the length of the list, i.e., the number of elements:

```
numElements = len(values)
```

Using the Square Brackets

- Note that there are two distinct uses of the square brackets. When the square brackets immediately follow a variable name, they are treated as the **subscript operator** (i.e., they identify an element of the list)

```
values[4]
```

- When the square brackets follow an “=” they **create** a list:

```
values = [4]
```

This statement creates a list with one element, the integer 4

Looping over Lists

- Given a list named “values” that contains 10 elements, we want to access each element of the list
- We can iterate over indices (with **range()**):

```
for i in range(len(values)) :  
    print(i, values[i])
```

- Or directly over values (if we don't need the index):

```
for elem in values:  
    print(elem)
```

A list is a
container

Same as with strings!

Looping over Lists (cont'd)

- You can also use the `enumerate` function to extract the (index, value) pairs:

```
for i, element in enumerate(values) :  
    print(i, element)
```

- In every iteration:
 - `i` assumes the value of an index in the list: 0, 1, 2, ...
 - `element` assumes the value of the corresponding element: `values[0]`, `values[1]`, `values[2]`, ...
- So, the code above is equivalent to the one of the previous slide:

```
for i in range(len(values)) :  
    print(i, values[i])
```

- The `enumerate` version is more «Pythonic», but also arguably more confusing for novices.
- `Enumerate`, can also be used for strings...

List References

- Make sure you understand the difference between the:
 - List **variable**: The named 'alias' (or **reference**) to the list
 - List **contents**: Memory where the values are stored

```
values = [32, 54, 67.5, 29, 35, 80, 115, 44.5, 100, 65]
```

List variable



Reference

List contents

[0]	10
[1]	9
[2]	7
[3]	4
[4]	5

Values

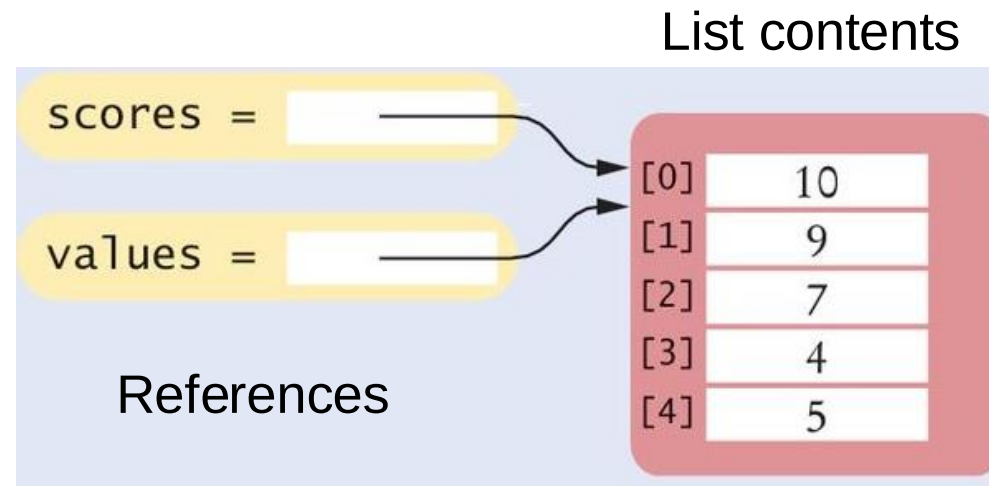
A list variable contains a **reference** to the list contents. The reference points to the **location** of the list contents (in memory).

List Aliases

- When you **copy** a list variable into another, both **variables** refer to **the same list**
 - The second variable is an alias for the first because both variables reference the same list

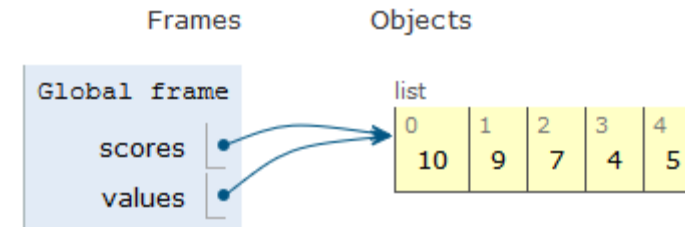
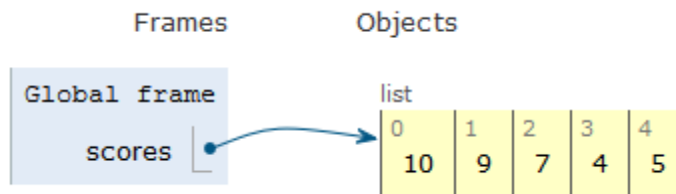
```
scores = [10, 9, 7, 4, 5]
values = scores      # Copying list reference
```

A list variable specifies the **location** of a list. Copying the reference yields a second **reference** to the **same** location, so to the same list.



Try it with PythonTutor

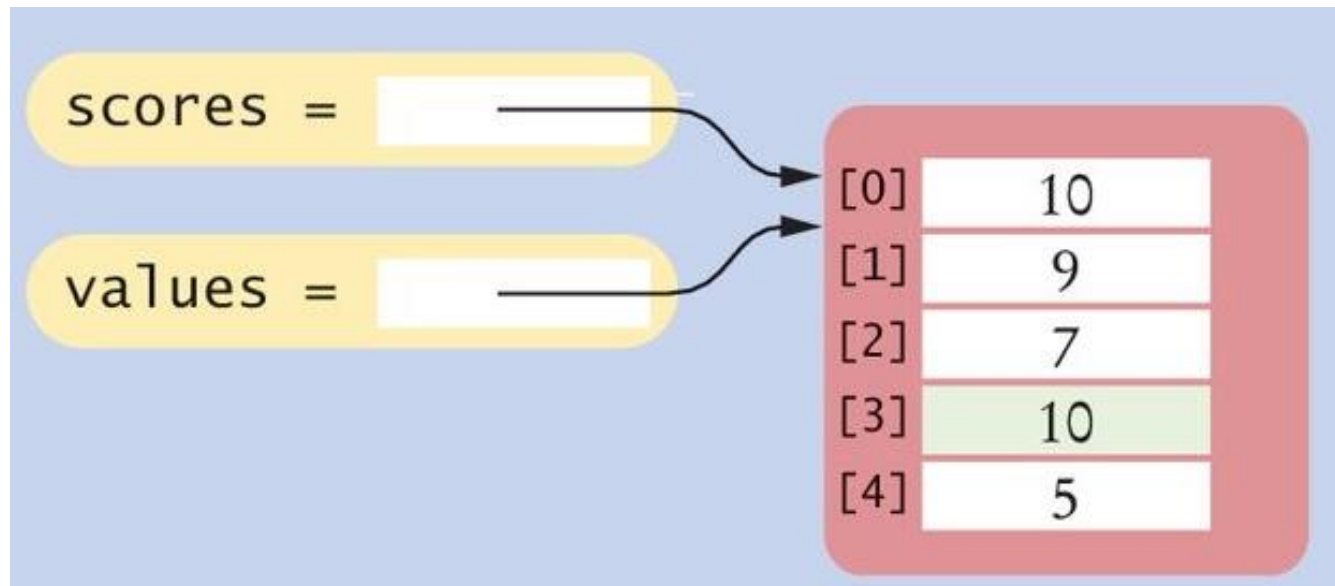
```
scores = [10, 9, 7, 4, 5]  
values = scores      # Copying list reference
```



Modifying Aliased Lists

- You can **modify** the list through either of the variables
- You are in fact modifying the **same** list (accessed with two different names)

```
scores[3] = 10  
print(values[3])    # Prints 10
```

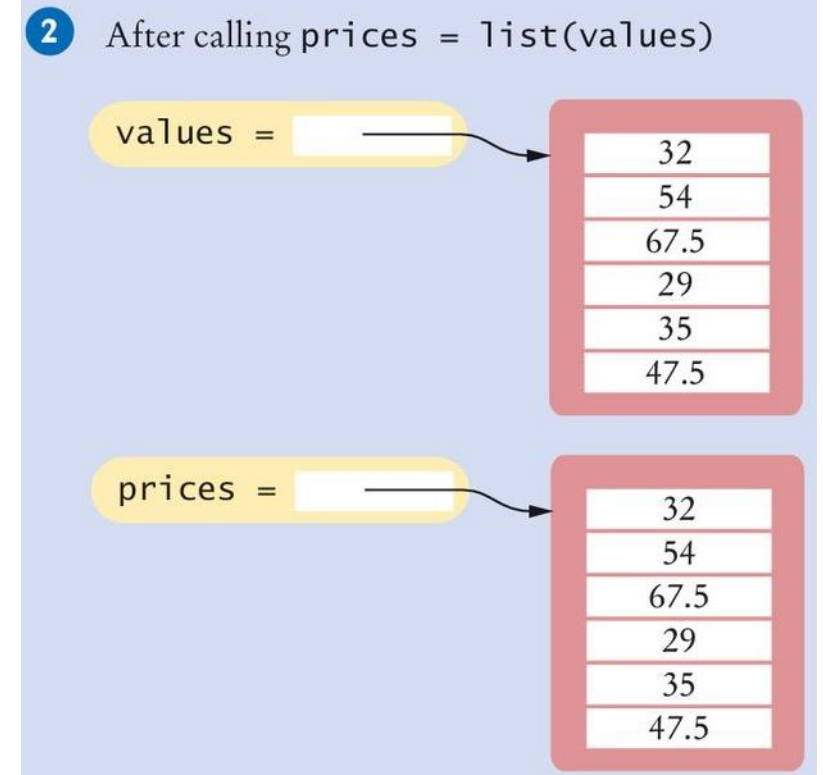


Copying Lists

- When you want to **create a copy** of a list, that is **a new list** with the same elements (in the same order) of the original one, you must use the **list()** function:

```
prices = list(values)
```

prices and **values** are now references to two different lists (unlike scores and values in the previous slide.)



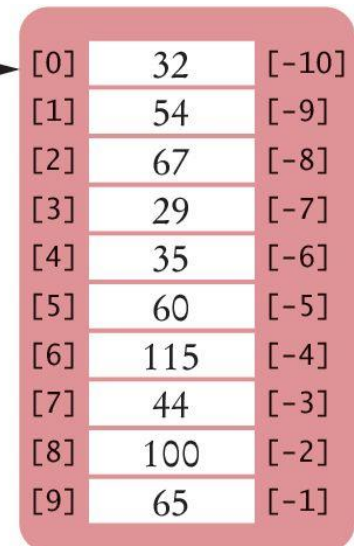
Negative/Reverse Subscripts

- Python, unlike other languages, allows **negative subscripts** to provide access to the list elements in **reverse** order.
 - For example, a subscript of `-1` provides access to the last element in the list:
 - Similarly, `values[-2]` is the second-to-last element.

```
last = values[-1]
print("The last element in
the list is", last)
```

```
# equivalent
last = values[len(values)-1]
```

values =



[0]	32	[-10]
[1]	54	[-9]
[2]	67	[-8]
[3]	29	[-7]
[4]	35	[-6]
[5]	60	[-5]
[6]	115	[-4]
[7]	44	[-3]
[8]	100	[-2]
[9]	65	[-1]

Same as strings!

List Operations



6.2

List Operations

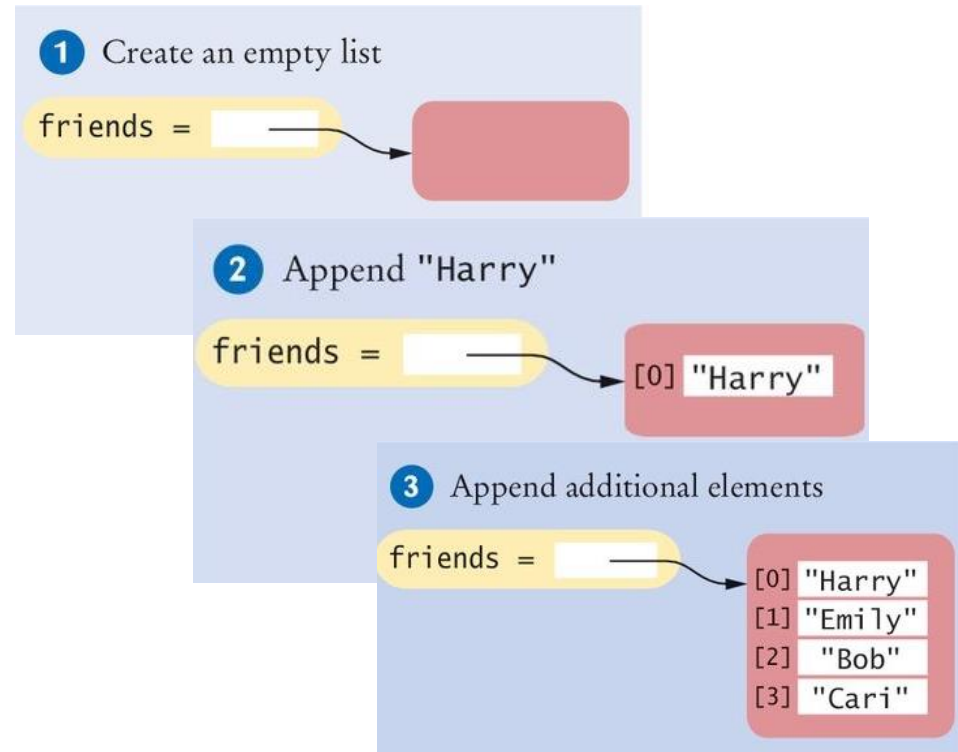
- Appending Elements
- Inserting an Element
- Finding an Element
- Removing an Element
- Concatenation
- Equality / Inequality Testing
- Sum, Maximum, Minimum, and Sorting

Try them with
PythonTutor!

Appending Elements

- We often need to add values to a list after it has been created
- In this case, we can create a (possibly empty) list and add elements to the end, as needed

```
#1  
friends = []  
  
#2  
friends.append("Harry")  
  
#3  
friends.append("Emily")  
friends.append("Bob")  
friends.append("Cari")
```



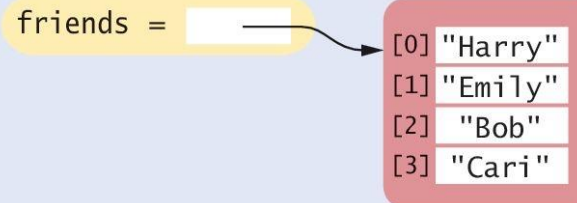
Inserting an Element

- Sometimes the order in which elements are added to a list is important
- We can **insert** new elements **at specific positions**
 - The other elements “move forward”

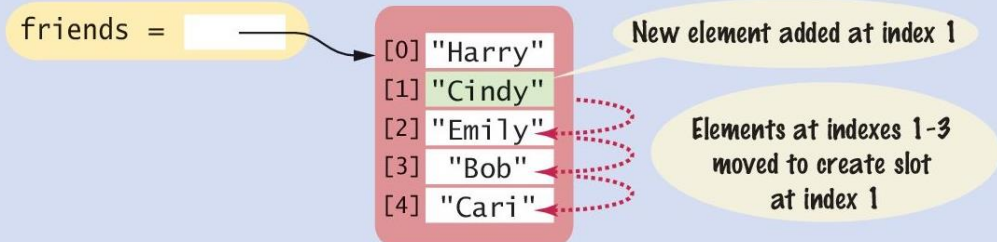
```
#1
friends = ["Harry", "Emily",
           "Bob", "Cari"]

#2
friends.insert(1, "Cindy")
```

1 The newly created list



2 After names.insert(1, "Cindy")



Finding an Element

- If you simply want to **know** whether an element **is present** in a list, use the **in** operator (**already seen for strings**):

```
if "Cindy" in friends :  
    print("She's a friend")
```

- The result is a Boolean value:
 - True if the element is contained in the list
 - False if the element is not contained in the list
 - Usually used in `while` or `if` statements

Finding an Element

- Often, you want to know the **position at which an element is found**
 - The `index()` method yields the index of the first match

```
friends = ["Harry", "Emily", "Bob", "Cari", "Emily"]  
n = friends.index("Emily") # Sets n to 1
```

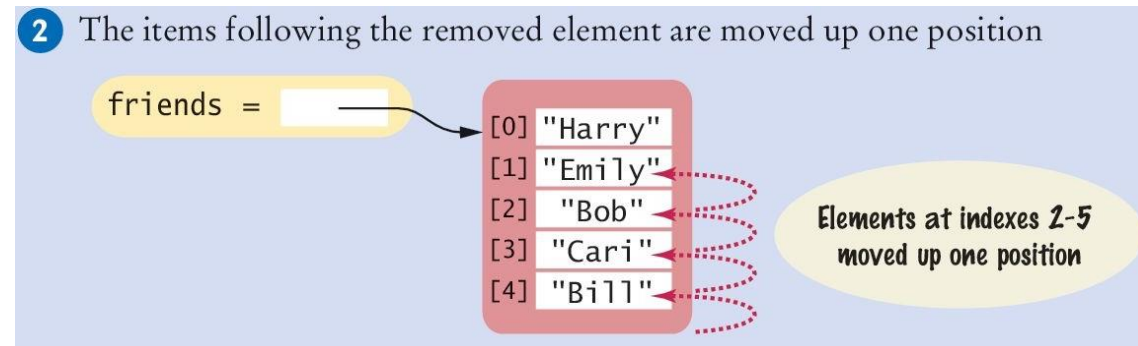
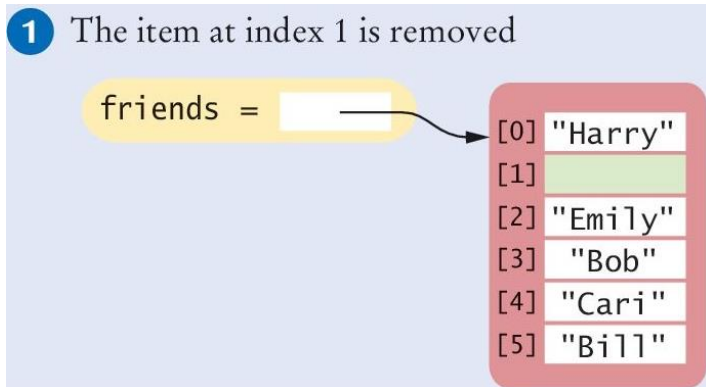
- If the element is **not** found in the list, a **ValueError** occurs
 - So, you better always first check if the element is present using the `in` operator.

Removing an Element by Index

- The `pop()` method removes the element **at a given position**

```
friends = ["Harry", "Cindy", "Emily", "Bob", "Cari", "Bill"]  
friends.pop(1)
```

- All of the elements following the removed element are moved by one position to “close the gap”
 - So, the length of the list is reduced by 1



Removing an Element by Value

- ✦ We can also remove an element based on **its value** (rather than its position) using the `remove()` method.

```
friends = ["Harry", "Cindy", "Emily", "Bob", "Cari", "Bill"]  
friends.remove("Cindy")  
# the list now contains  
# ["Harry", "Emily", "Bob", "Cari", "Bill"]
```

- ✦ As with `index()`, we better first check if the element is there, to avoid errors:

```
If "Cindy" in friends:  
    friends.remove("Cindy")
```

Better!

Concatenation

- The **concatenation** of two lists is a new list that contains the elements of the first list, followed by the elements of the second
- Two lists can be concatenated by using the plus (+) operator (**same as strings**):

```
myFriends = ["Fritz", "Cindy"]  
yourFriends = ["Lee", "Pat", "Phuong"]
```

```
ourFriends = myFriends + yourFriends  
# Sets ourFriends to ["Fritz", "Cindy", "Lee", "Pat", "Phuong"]
```

Replication

- **Replication** of a list generates many copies of the elements of a list (similar to string replication)

```
monthInQuarter = [ 1, 2, 3 ] * 4
```

- Results in the list [1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
- The **integer** specifies how many copies of the list should be concatenated
- One common use of replication is to initialize a list with a fixed value

```
monthlyScores = [0] * 12
```

A list with 12 zeros

Equality / Inequality Testing

- You can use the `==` operator to compare whether two lists have the same elements, in the same order

```
[1, 4, 9] == [1, 4, 9]    # True  
[1, 4, 9] == [4, 1, 9]   # False.
```

- The opposite of `==` is `!=`

```
[1, 4, 9] != [4, 9]      # True.
```

Sum, Maximum, Minimum

- If you have a list of numbers, the `sum()` function yields the sum of all values in the list.

```
sum([1, 4, 9, 16]) # Yields 30
```

- Error if the list contains strings or other non-numeric types...
- For a list of numbers *or strings*, the `max()` and `min()` functions return the largest and smallest value:

```
max([1, 16, 9, 4])           # Yields 16  
min("Fred", "Ann", "Sue")    # Yields "Ann"
```

- Error if the list contains a **mix** of strings and numbers...

Sorting

- The `sort()` method sorts a list of numbers or strings, from the smallest to the largest value
- The sort happens “in place”, within the same list

```
values = [1, 16, 9, 4]  
values.sort() # Now values is [1, 4 , 9, 16]
```

- The `sorted()` function, instead, creates **a new sorted list, without modifying the original one**

```
values = [1, 16, 9, 4]  
other = sorted(values)  
# other contains [1, 4 , 9, 16] and values is unchanged
```

Sorting (Cont'd)

- By default, the `sort()` method and the `sorted()` function sort the elements from **smallest to largest**.
- If you want the inverse order (largest to smallest) you can use the optional parameter `reverse`

```
values = [1, 16, 9, 4]
values.sort(reverse=True) # values now contains [16, 9, 4, 1]
```

```
values = [1, 16, 9, 4]
other = sorted(values, reverse=True)
# other now contains [16, 9, 4, 1]
# values is unchanged
```

Slices of a List

- We can use slicing to **extract a part of a list** (similar to what we've seen for strings):
- Example: list of temperatures, one per month

```
temperatures = [18, 21, 24, 33, 39, 40, 39, 36, 30, 22, 18]  
# extracts the temperatures of the third quarter, as a new list!!  
thirdQuarter = temperatures[6 : 9]
```

- The two numbers are the first index (included) and last index (excluded).
 - So, in our example we get elements 6, 7, and 8

Slices (2)

- Both indexes used with the slice operator are optional
 - If the first index is omitted, the slice implicitly starts from the **start of the list**
 - If the second index is omitted, the slice implicitly ends at the **end of the list**

- Examples

```
x = temperatures[ : 6]
```

```
# x includes all elements up to, but not including,
```

```
# position 6
```

```
x = temperatures[6 : ]
```

```
# includes all elements starting at position 6 to the end
```

```
# of the list
```

Slices (3)

- We can also omit both:

```
x = temperatures[ : ]
```

```
# all elements from start to end. Equivalent to making
```

```
# a copy with x = list(temperatures)
```

- A third index can be used to specify the step (implicitly = 1)
- `temperatures[::2]`
 - All elements with **even** indices
- `temperatures[::-1]`
 - The entire list, but in **reverse** order

Slices (4)

- With lists, you can use slicing for **assignments** too:

`temperatures[6 : 9] = [45, 44, 40]`

- Replaces the values in elements 6, 7, and 8
 - Other elements are unaffected

**Not possible with
strings (immutable)**

- **Note: if the length of the replacement is different from the length of the slice, elements will be removed or added to the list**

Common List Functions And Operators

Table 1 Common List Functions and Operators

Operation	Description
<code>[]</code> <code>[<i>elem</i>₁, <i>elem</i>₂, ..., <i>elem</i>_{<i>n</i>}]</code>	Creates a new empty list or a list that contains the initial elements provided.
<code>len(<i>l</i>)</code>	Returns the number of elements in list <i>l</i> .
<code>list(<i>sequence</i>)</code>	Creates a new list containing all elements of the sequence.
<code>values * num</code>	Creates a new list by replicating the elements in the values list <i>num</i> times.
<code>values + moreValues</code>	Creates a new list by concatenating elements in both lists.

Common List Functions And Operators (2)

Table 1 Common List Functions and Operators

Operation	Description
$l[\text{from} : \text{to}]$	Creates a sublist from a subsequence of elements in list l starting at position from and going through but not including the element at position to . Both from and to are optional. (See Special Topic 6.2.)
$\text{sum}(l)$	Computes the sum of the values in list l .
$\text{min}(l)$ $\text{max}(l)$	Returns the minimum or maximum value in list l .
$l_1 == l_2$	Tests whether two lists have the same elements, in the same order.

Common List Methods

Table 2 Common List Methods	
Method	Description
<i>l.pop()</i> <i>l.pop(position)</i>	Removes the last element from the list or from the given position. All elements following the given position are moved up one place.
<i>l.insert(position, element)</i>	Inserts the <i>element</i> at the given position in the list. All elements at and following the given position are moved down.
<i>l.append(element)</i>	Appends the element to the end of the list.
<i>l.index(element)</i>	Returns the position of the given element in the list. The element must be in the list.
<i>l.remove(element)</i>	Removes the given element from the list and moves all elements following it up one position.
<i>l.sort()</i>	Sorts the elements in the list from smallest to largest.

List Methods

Python List Methods

append() - Add an element to the end of the list

extend() - Add all elements of a list to the another list

insert() - Insert an item at the defined index

remove() - Removes an item from the list

pop() - Removes and returns an element at the given index

clear() - Removes all items from the list

index() - Returns the index of the first matched item

count() - Returns the count of the number of items passed as an argument

sort() - Sort items in a list in ascending order

reverse() - Reverse the order of items in the list

copy() - Returns a shallow copy of the list

<https://www.programiz.com/python-programming/list>

Common List Algorithms



6.3

Common List Algorithms

- Filling a List
- Combining list elements (“reduction”)
- Maximum and Minimum
- Linear Search
- Collecting and Counting Matches
- Removing Matches
- Swapping Elements
- Reading Input

Filling a List

- We can fill a list using a **loop** (based on the criterion determined by the specific problem) and the **append()** method.
- Example: this code **creates** and **fills** a list with squares of integer numbers (0, 1, 4, 9, 16, ...)

```
values = []  
for i in range(n) :  
    values.append(i * i)
```

Filling a List (with User Input)

- It's common to fill a list using inputs from the user:

```
values = []
userInput = input("Please insert values, Enter to quit:")
while userInput != "":
    values.append(float(userInput))
    userInput = input("Please insert values, Enter to quit:")
```

```
Please insert values, Enter to quit:
32
29
67.5
```


Combining List Elements

- Summing the elements of a list, without using **sum()**

```
result = 0.0
for element in values :
    result = result + element
```

- Given a list of strings, create a single string that **concatenates** all the list elements:

```
result = ""
for element in names :
    result = result + element
```

Element Separators

- When you display the elements of a list of strings, you usually want to separate them, often with commas or vertical lines, like this:

Harry, Emily, Bob

- Add the separator **before each** element, **except the initial one** (with index 0), like this:

```
result = ''
for i in range(len(names)) :
    if i > 0 :
        result = result + ", "
    result = result + names[i]
print(result)
```

Python Shortcut: `.join()`

- The `.join` method of strings automatically joins the elements of a list, using the string as a separator

`separator_string.join(list)`

- Example (this produces the same result as the code in the previous slide):

```
result2 = ', '.join(names)
print(result2)
```

Python Shortcut: `.split()`

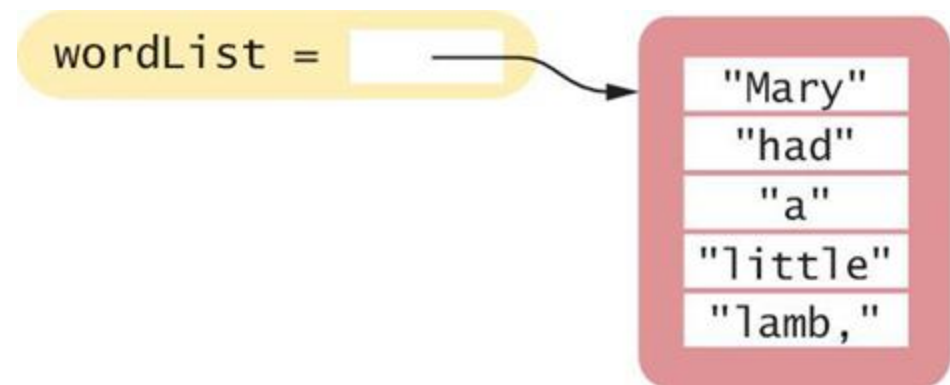
- The `split(sep)` of a **string** returns a **list** of the **substrings** resulting from splitting the original string in correspondence of each occurrence of the separator
 - If no separator is specified, the string is split after every occurrence of **blank** characters

- Example:

```
line = M a r y   h a d   a   l i t t l e   l a m b ,
```

```
wordList = line.split()
```

- Creates a list containing 5 strings, in the same order in which they appear in the original string.



Python Shortcut: `.split()`

```
str.split(sep=None, maxsplit=-1)
```

Return a list of the words in the string, using *sep* as the delimiter string. If *maxsplit* is given, at most *maxsplit* splits are done (thus, the list will have at most *maxsplit*+1 elements). If *maxsplit* is not specified or `-1`, then there is no limit on the number of splits (all possible splits are made).

If *sep* is given, consecutive delimiters are not grouped together and are deemed to delimit empty strings (for example, `'1,,2'.split(',')` returns `['1', '', '2']`). The *sep* argument may consist of multiple characters (for example, `'1<>2<>3'.split('<>')` returns `['1', '2', '3']`). Splitting an empty string with a specified separator returns `['']`.

For example:

```
>>> '1,2,3'.split(',')
['1', '2', '3']
>>> '1,2,3'.split(',', maxsplit=1)
['1', '2,3']
>>> '1,2,,3'.split(',')
['1', '2', '', '3', '']
```

If *sep* is not specified or is `None`, a different splitting algorithm is applied: runs of consecutive whitespace are regarded as a single separator, and the result will contain no empty strings at the start or end if the string has leading or trailing whitespace. Consequently, splitting an empty string or a string consisting of just whitespace with a `None` separator returns `[]`.

For example:

```
>>> '1 2 3'.split()
['1', '2', '3']
>>> '1 2 3'.split(maxsplit=1)
['1', '2 3']
>>> ' 1 2 3 '.split()
['1', '2', '3']
```

Maximum and Minimum

- Here is the implementation of the max and min algorithms:

```
largest = values[0]
for i in range(1, len(values)) :
    if values[i] > largest :
        largest = values[i]
```

```
# equivalent to
largest = max(values)
```

```
smallest = values[0]
for i in range(1, len(values)) :
    if values[i] < smallest :
        smallest = values[i]
```

```
# equivalent to
smallest = min(values)
```

Maximum and Minimum

- Note that simple `max()/min()` cannot be used if we need to find the max/min based on some characteristic of our elements.
- Example: in a list of strings, we look for the **longest string**:
 `names = ["Ann", "Charlotte", "Zachary", "Bill"]`
 - `max(names)` does something else...
 - We need to use the pattern seen in the previous slide

```
longest = names[0]
for i in range(1, len(names)) :
    if len(names[i]) > len(longest):
        longest = names[i]
```

Linear Search

- Example: find the first value that is > 100
 - In general, the first element that satisfies a certain condition....
- You need to visit all elements until you find a match or you reach the end of the list:

```
limit = 100
pos = 0
found = False
while pos < len(values) and not found :
    if values[pos] > limit :
        found = True
    else :
        pos = pos + 1
if found :
    print("Found at position:", pos)
else :
    print("Not found")
```

A linear search
inspects elements
in sequence until a
match is found.

Collecting and Counting Matches

- Collecting all matches

```
result = []  
for element in values :  
    if # element respect a certain condition :  
        result.append(element)
```

- Counting matches

```
counter = 0  
for element in values :  
    if # element respect a certain condition  
        counter = counter + 1
```

Removing Matches

- Remove all elements that match a particular condition
 - Example: remove all strings of length < 4 from a list

```
i = 0
while i < len(words) :
    word = words[i]
    if len(word) < 4 :
        words.pop(i) # delete i-th element
        # do NOT increment i
    else :
        i = i + 1
```

Removing matches (cont'd)

- Why can't we just write:

```
for w in words:  
    if len(word) < 4 :  
        words.remove(w) # cancellare w
```

- Simpler, more readable...
- **But Wrong!!!!..... Why ?**

Exercise 2

✦ Write a program that eliminates all occurrences of a certain value from a list

✦ Example: original list:

- $L = [2, 3, 2, 4, 2, 2, 5, 2, 2, 5, 6, 4]$

- Remove all occurrences of value 2. Final list:

- $L = [3, 4, 5, 5, 6, 4]$

Checking conditions

- Two common cases:

All elements satisfy a condition C

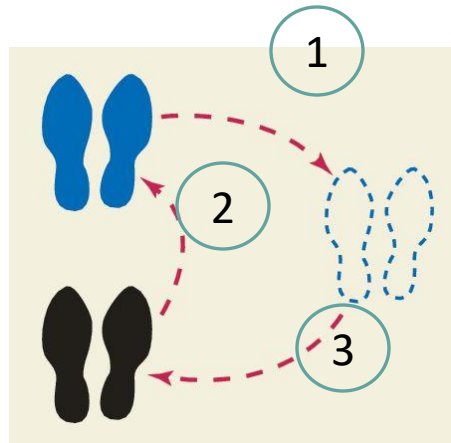
```
flag = True
while i < len(L) and flag:
    if L[i] does not satisfy C:
        flag = False
    else :
        i = i + 1
if flag:
    print("Condition verified")
else :
    print("Condition not verified")
```

At least one element satisfies a condition C

```
flag = False
while i < len(L) and not flag:
    if L[i] satisfies C:
        flag = True
    else :
        i = i + 1
if flag:
    print("Condition verified")
else :
    print("Condition not verified")
```

Swapping Elements

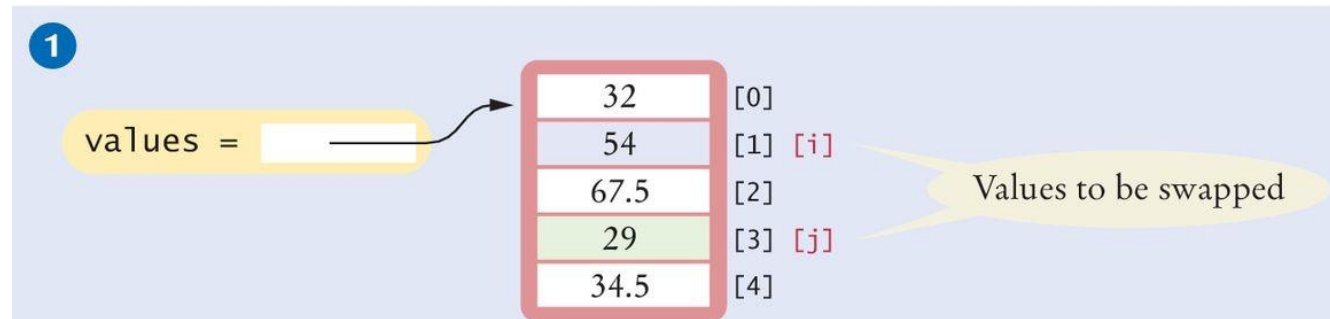
- **Swap the elements** at positions i and j of a list
 - We'd like to set $l[i]$ to $l[j]$.
 - Then we want to set $l[j]$ to $l[i]$
 - ...but in the previous step, we have overwritten (and lost) the original value of $l[i]$
 - We should have **saved** the original value...
 - Swapping two variables requires a **third, temporary variable...**



Before moving a new value into a location (say blue) copy blue's value elsewhere and then move black's value into blue. Then move the temporary value (originally in blue) into black.

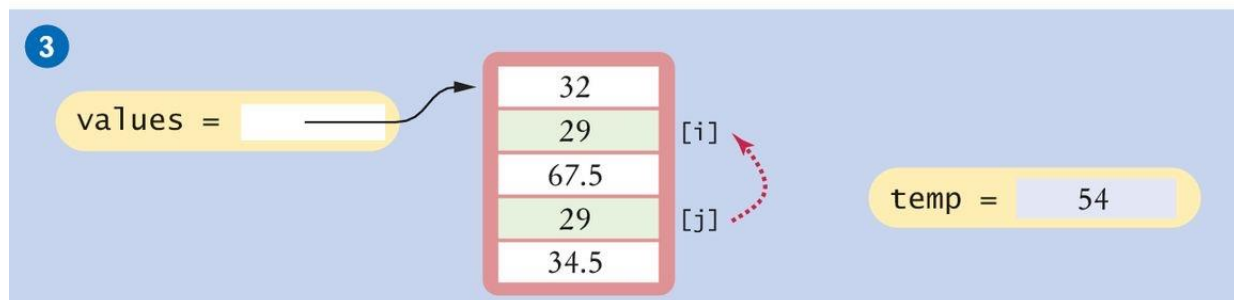
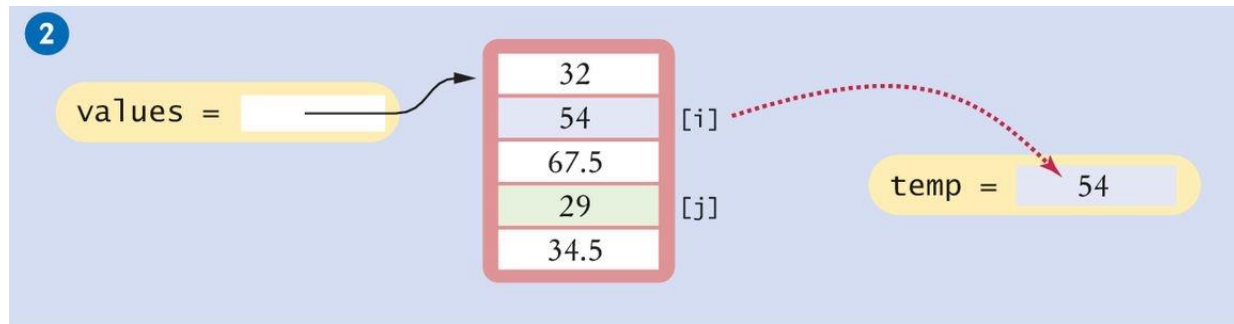
Swapping Elements (2)

- Swapping elements [1] and [3]
 - This sets up the scenario for the actual code that will follow



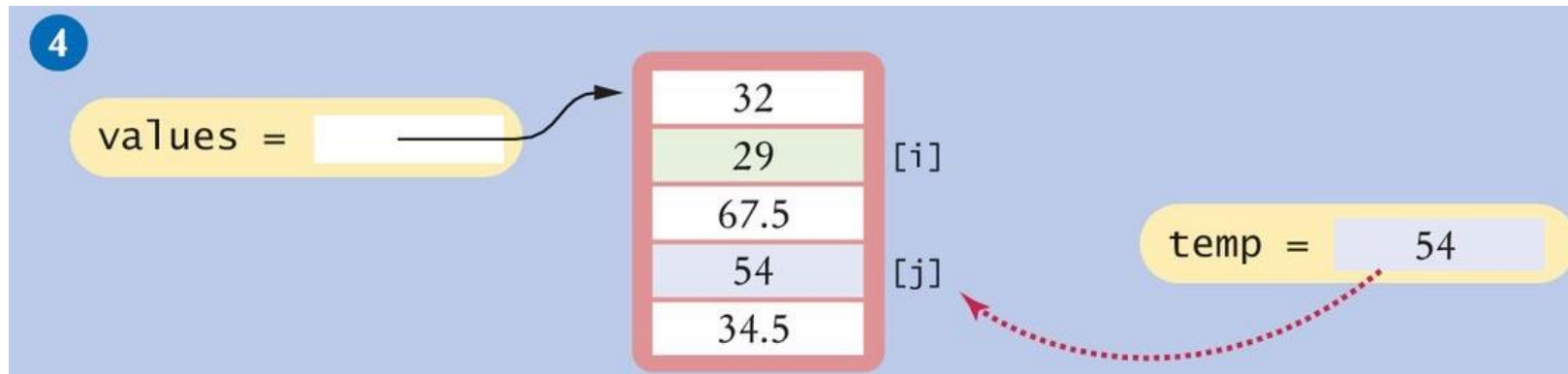
Swapping Elements (3)

```
# Step 1  
temp = values[i]  
  
# Step 2  
values[i] = values[j]
```



Swapping Elements (4)

```
# Step 3  
# temp contains the “original” values[i]  
values[j] = temp
```



Exercise (P6.30)

- ★ Given two lists **a** e **b** create a new list that alternates elements from **a** elements from **b**.
- ★ **Careful: the two lists do not have necessarily the same length**
 - If one list is shorter than the other, the elements must be alternated as long as this is possible, then the remaining elements of the longest list must be appended (in order) to the end of the list.
- ★ Example:
- ★ **a = [1, 4, 9, 16]**
- ★ **b = [9, 7, 4, 9, 11, 2]**
- ★ The new list shall contain the values: **[1, 9, 4, 7, 9, 4, 16, 9, 11, 2]**

Using Lists With Functions



6.4

Using Lists With Functions

- A function can accept **a list as an argument**
 - Nothing special, except if we **modify the list contents from inside the function**.
- The following function visits the list elements, but it does not modify them

```
def sumsq(values) :  
    total = 0  
    for element in values :  
        total = total + element**2  
    return total
```

Try it with
PythonTutor!

Modifying List Elements

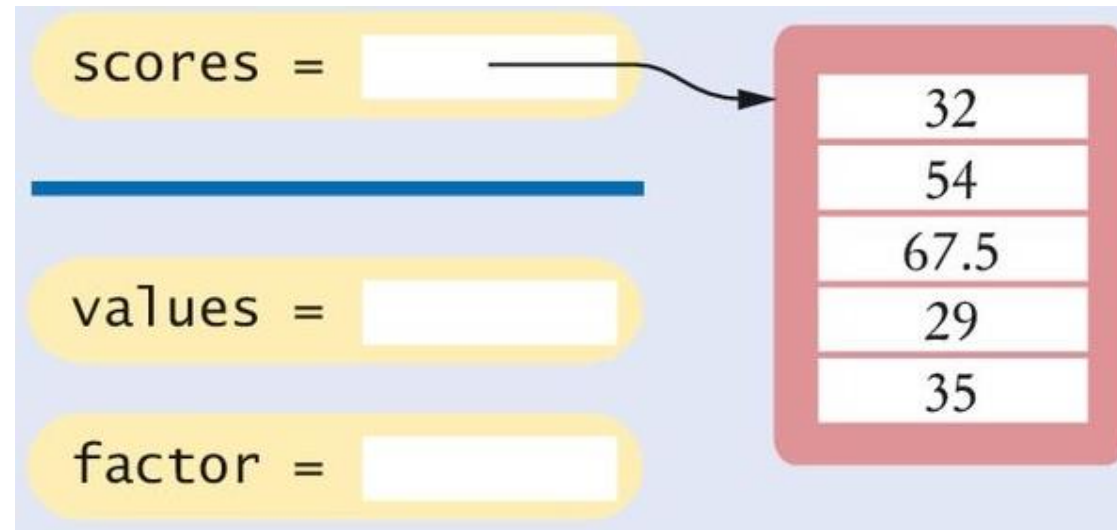
- The following function **multiplies all elements of a list** by a given factor
- **CAREFUL: The original list is modified!!**
 - Remember: the list name (the parameter that we're passing to the function) is a reference to the "location" of the list.

```
def multiply(values, factor) :  
    for i in range(len(values)) :  
        values[i] = values[i] * factor
```

Try it with
PythonTutor!

Example: Step 1

- Outside the function, we have a list variable called **scores**

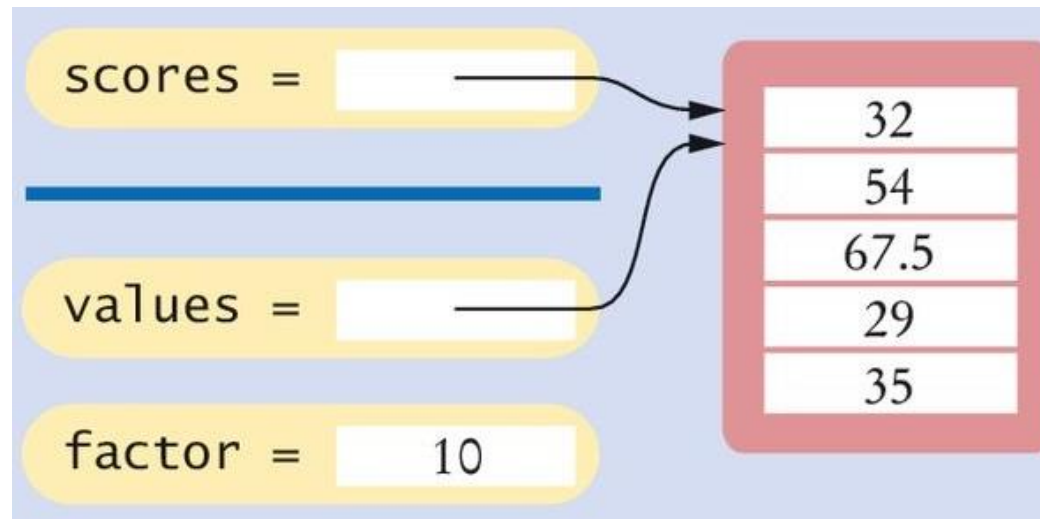


Example: Step 2

- The function parameters are initialized with the arguments that are passed in the call

```
# Function call  
multiply(scores, 10)
```

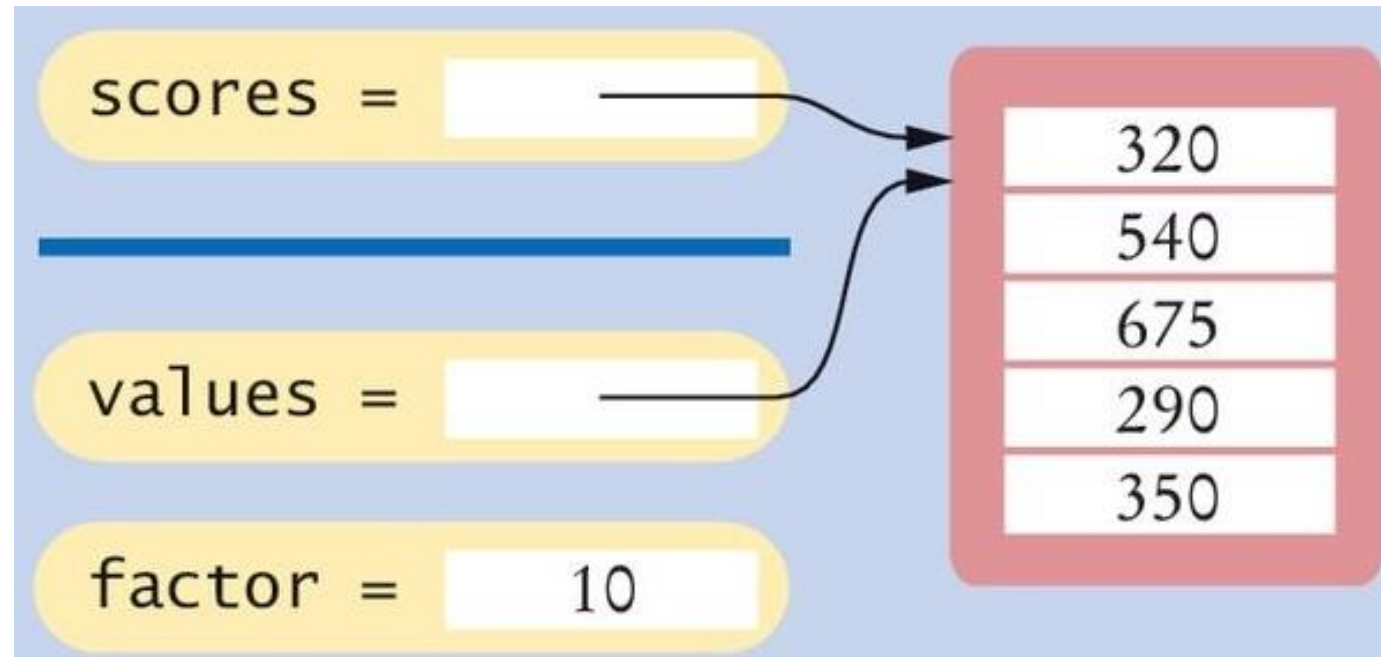
- In our case, `values` is set to `scores` and `factor` is set to `10`
 - Note that `values` and `scores` are [references to the same list](#)



Example: Step 3

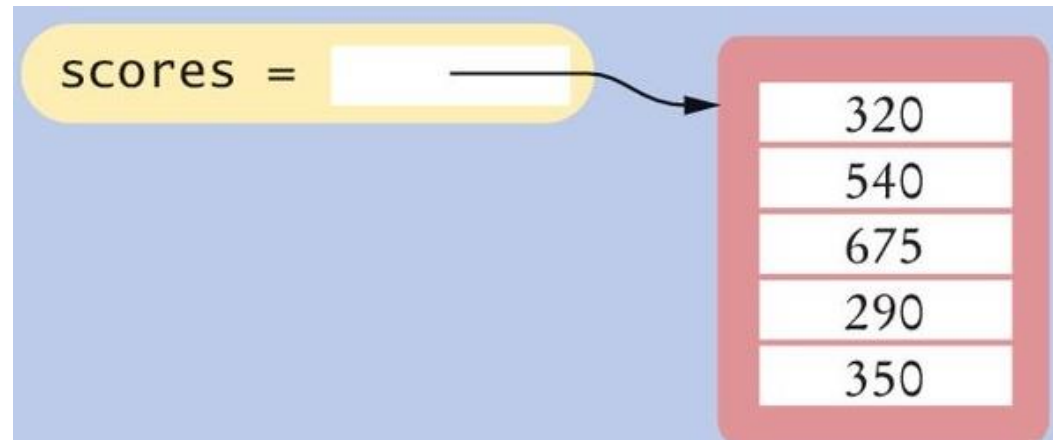
- The function multiplies all list elements by 10

```
def multiply(values, factor) :  
    for i in range(len(values)) :  
        values[i] = values[i] * factor
```



Example: Step 4

- The function **returns**. Its parameter variables are removed
- However, `scores` still refers to the list with the modified elements



Returning Lists From Functions

- A function can return a list.
- Simply create the list in the function and return it
- In this example, the `squares()` function returns a list of squares from 0^2 up to $(n - 1)^2$

```
def squares(n) :  
    result = []  
    for i in range(n) :  
        result.append(i * i)  
    return result
```

Example

- Write a program that reads values from the user, multiplies them by 10, and prints them in reverse order, using three functions
- The readFloats function returns a list of values read from the user.
- The multiply function has a list argument, it modifies the list elements
- The printReversed function has a list argument, but it does not modify the list elements

Tuples



6.4

Tuples

- A tuple is similar to a list, but once created, its contents cannot be modified (a tuple is an **immutable version of a list**).
- A tuple is created by specifying its contents as a comma-separated sequence. You can enclose the sequence in () parentheses

```
triple = (5, 10, 15)
```

- If you prefer, you can omit the parentheses:

```
triple = 5, 10, 15
```

- Note: [tuples](#) are an advanced data structure, with many more characteristics. They will [not](#) be studied in depth in this course.

Tuple operations

- Tuples share many of the same operations used for lists

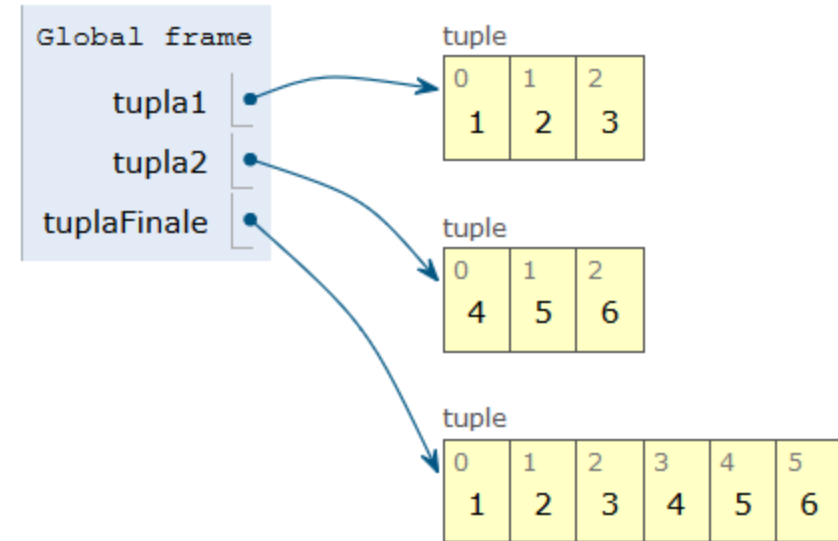
- Access to elements with `[]`
- Concatenation with `+`

```
tupla1 = (1, 2, 3)
```

```
tupla2 = (4, 5, 6)
```

```
tuplaFinale = tupla1 + tupla2
```

- `len()`, `sum()`, `max()` functions
- Iteration using `for`
- The `in` and `not in` operators



Tuple operations

- Of course, being immutable, we cannot assign tuple elements after its creation:

```
>>> myTuple = (1, 2, 3)
>>> myTuple[0] = 5
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> █
```

Python Shortcut: multiple assignment

- More than one assignment may be done **in parallel** (and we avoid the temp variable) by using a **tuple**
 - `(a, b) = (3, 4)` is equivalent to `a=3` and `b=4`, simultaneously
 - `(a, b) = (b, a)` effectively swaps `a` and `b`

```
# shortcut for swap  
  
( values[1], values[3] ) = ( values[3], values[1] )
```

- Can be used to swap two variables without a third support variable.

Returning Multiple Values

- It is common practice in Python to use tuples to return multiple values from a function

```
# Function definition
def readDate() :
    print("Enter a date:")
    month = int(input(" month: "))
    day = int(input(" day: "))
    year = int(input(" year: "))
    return (year, month, day) # Returns a tuple.

# Function call: assign entire value to a tuple
date = readDate()

# Function call: use tuple assignment:
(year, month, day) = readDate()
```

The enumerate() function (reprise)

- ★ The enumerate function turns **any container** into a *sequence of tuples*

- `name = "ciao"`



- `enumerate(name)`



- `for (index, value) in enumerate(name):`

- ★ **Note:** the return value of `enumerate()` is **NOT a list**, but a particular “enumerate” object (you can turn it into a list with `list()`)

Adapting Algorithms



6.5

Adapting Algorithms

- We've seen a good number of “standard algorithms” using lists
- But real problems often require to **adapt** (or at least combine) these algorithms, changing them slightly to suit our needs...

Example

- You are given the quiz scores of a student. You must compute the final quiz score, which is **the sum of all scores after dropping the lowest one**

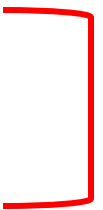
- For example, if the scores are

8 7 8.5 9.5 7 5 10

- then the final score is 50

Variant

- Problem Statement: The final quiz score for a student is computed by adding up all of the scores **except the lowest two**
 - For example, if the scores are: 8, 4, 7, 9, 9, 7, 5, 10
 - The final score is 50
- Solution [try at home]:
 - Read inputs
 - Remove the **minimum**
 - Remove the minimum **again**
 - Calculate the **sum**



Same thing twice? → Use a function

Testing

- Develop your test cases, and the expected outputs

Test Case	Expected Output	Comment
8 4 7 8.5 9.5 7 5 10	50	Example case
8 7 7 7 9	24	Make sure only two instances of the low score are removed
8 7	0	After removing the two low scores, none remain
8	Error	At least 2 scores are needed
(no inputs)	Error	That is not a legal input

A Second Example

- Problem Statement: Our task is to analyze whether a dice is **fair** by **counting** how often each value (1, 2, 3, 4, 5, 6) appears
- Our input will be a series of dice toss values
 - For example, if the values are: 1, 2, 1, 3, 4, 6, 5, 6
 - The result is 1: 2; 2: 1; 3: 1; 4: 1; 5: 1; 6: 2
- We are going develop the algorithm and write a program to compute and print the frequency of each dice value

Try to solve it before looking at the following slides

Step One

- We want to start with a high level decomposition of the problem:
 - Read the dice values
 - Count how often the values (1, 2, ..., 6) appear
 - Print the counts
- If we think about this we can simplify; do we **need** to store the values?
 - We are only counting the number of times each dice toss occurs. If we create **a list of counters** we can read and then discard the inputs
- Our next step in the stepwise refinement is to identify the steps we need to process the data:
 - Read input
 - For each input value:
 - Increment the corresponding counter
 - Print the counters

Step Two

- Determine the algorithms we need
- We don't have an algorithm for reading inputs and incrementing a counter (yet) but it is easy to build one
 - If we have a list of length 6 we can simply
$$\text{counters}[\text{value} - 1] = \text{counters}[\text{value} - 1] + 1$$
- We therefore define $\text{counters} = [0] * \text{sides}$
- Now we can focus on printing the counters
- We can use a count-controlled loop and a format string to print the results

Step Three

- Plan the Functions we need:
 - `countInputs(sides)` # will count the inputs
 - `printCounters(counters)` # will print the counters
- The main function calls these functions:
`counters = countInputs(6)`
`printCounters(counters)`

Step Four

- Assemble and test your program:
- When updating a counter we have to make sure we do not generate a boundary error; we have to reject inputs < 1 and > 6

Test Case	Expected Output	Comment
1 2 3 4 5 6	1 1 1 1 1 1	Each number occurs once
1 2 3	1 1 1 0 0 0	Numbers that do not appear have a count of "0"
1 2 3 1 2 3 4	2 2 2 1 0 0	The counters must be correct
No input	0 0 0 0 0 0	All counters are "0"
0 1 2 3 4 5 6 7	ERROR	Inputs out of bounds

Extra

- **Try at home:** Format the program output as a histogram

```
1 : ( 16.1% ) +++++  
2 : ( 16.8% ) +++++  
3 : ( 17.7% ) +++++  
4 : ( 17.0% ) +++++  
5 : ( 16.4% ) +++++  
6 : ( 16.0% ) +++++
```

- **NOTE:** This is an example of a common problem “computing occurrences of a set of values”. Other examples:
 - How often each letter of the alphabet appears in a text
 - How many words of length 1,2, 3, ...etc, are present in a string
 - Many others..

Tables



6.7

Tables

- Lists can be used to store data in two dimensions (2D) like a spreadsheet

- Rows and Columns
- Also known as a '**matrix**'

- Technically, we will build tables as **lists of lists**:

- **Lists whose elements are other lists**

	Gold	Silver	Bronze
Canada	0	3	0
Italy	0	0	1
Germany	0	0	1
Japan	1	0	0
Kazakhstan	0	0	1
Russia	3	1	1
South Korea	0	1	0
United States	1	0	1

Figure 10 Figure Skating Medal Counts

Creating Tables

- This code creates a table that contains 8 rows and 3 columns, which is suitable for holding our medal count data:

Every element is a list

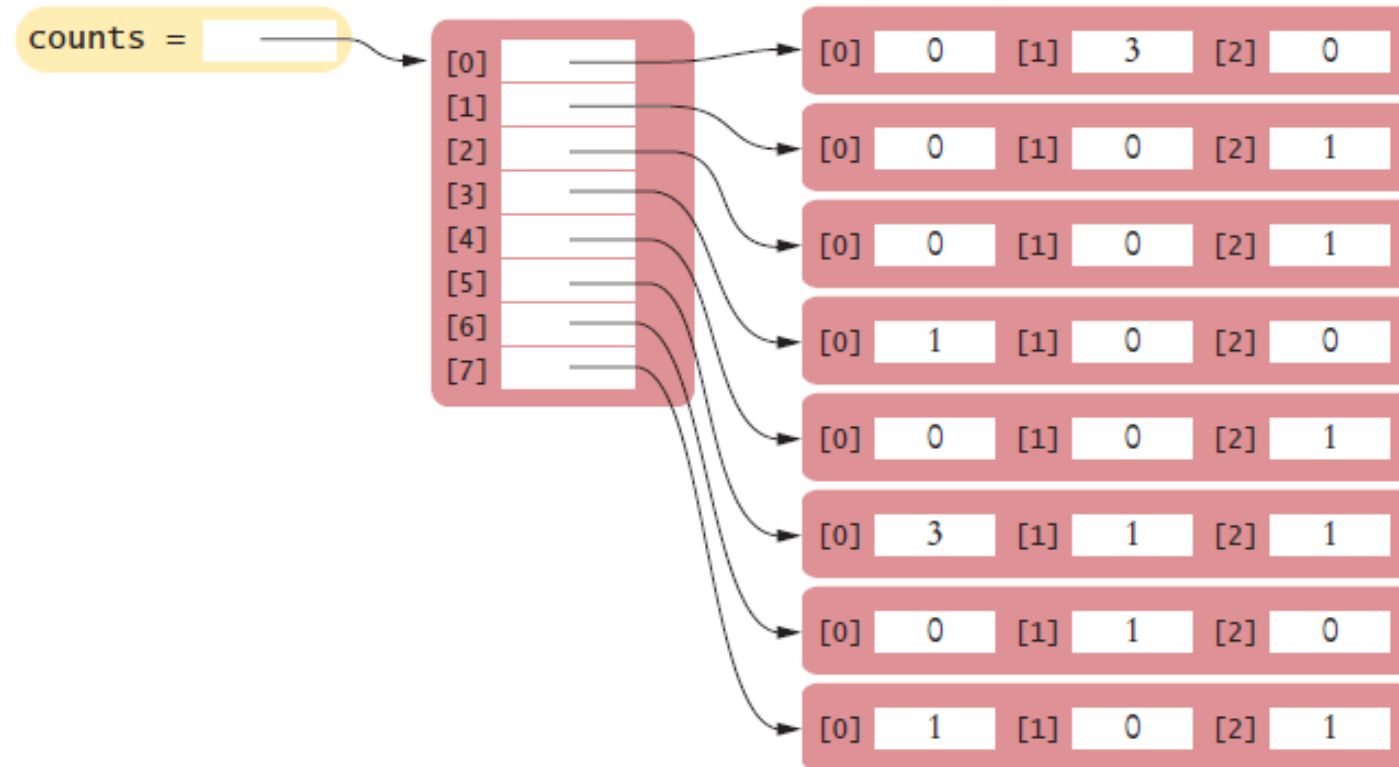
```
COUNTRIES = 8
MEDALS = 3

counts = [
    [ 0, 3, 0 ],
    [ 0, 0, 1 ],
    [ 0, 0, 1 ],
    [ 1, 0, 0 ],
    [ 0, 0, 1 ],
    [ 3, 1, 1 ],
    [ 0, 1, 0 ],
    [ 1, 0, 1 ]
]
```

Try it with
PythonTutor!

Creating Tables (2)

- It creates a list in which each element is itself another list:

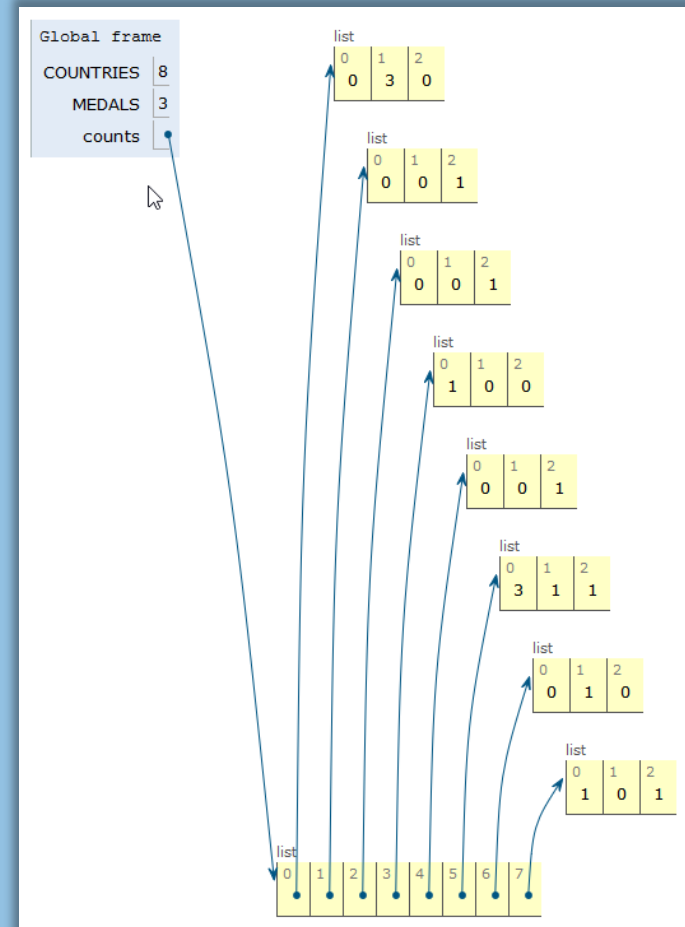


What really happens

```
COUNTRIES = 8
```

```
MEDALS = 3
```

```
counts = [  
    [ 0, 3, 0 ],  
    [ 0, 0, 1 ],  
    [ 0, 0, 1 ],  
    [ 1, 0, 0 ],  
    [ 0, 0, 1 ],  
    [ 3, 1, 1 ],  
    [ 0, 1, 0 ],  
    [ 1, 0, 1 ]  
]
```



Creating Tables (3)

- Sometimes, you may need to create a table with a size that is too large to initialize it with literal values
 - First, create an empty list that will be used to store the individual rows:

```
table = []
```

- Then, use a loop to create **each row of the table**, as a list with a number of elements equal to the number of columns.

```
ROWS = 5
COLUMNS = 20
for i in range(ROWS) :
    row = [0] * COLUMNS
    table.append(row)
```

5x20 table filled with 0s

- Or... **compact version**:
 - Only works for constant initialization

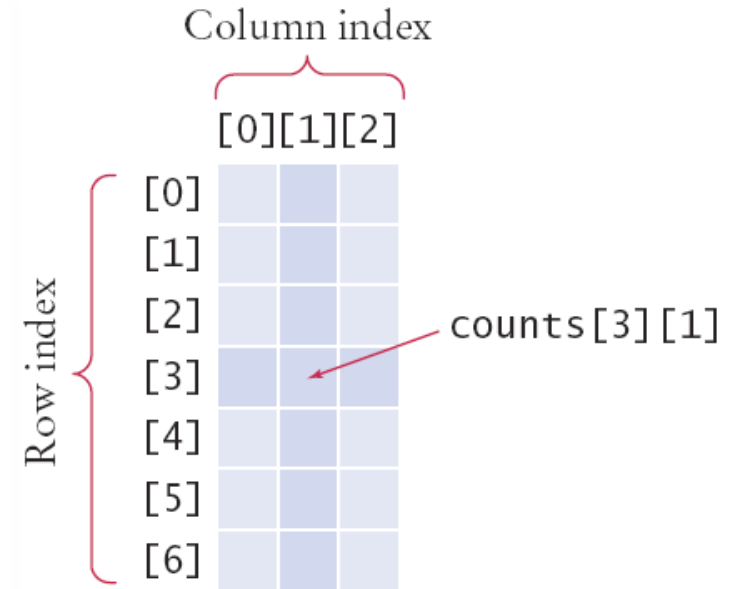
```
ROWS = 5
COLUMNS = 20
table = [[0]*COLUMNS]*ROWS
```

Accessing Elements

- Use two index values:
 - First the **Row** index, then the **Column** index

```
medalCount = counts[3][1]
```

- To print
 - Use nested for loops
 - Outer loop on row (*i*) , inner loop on column (*j*)

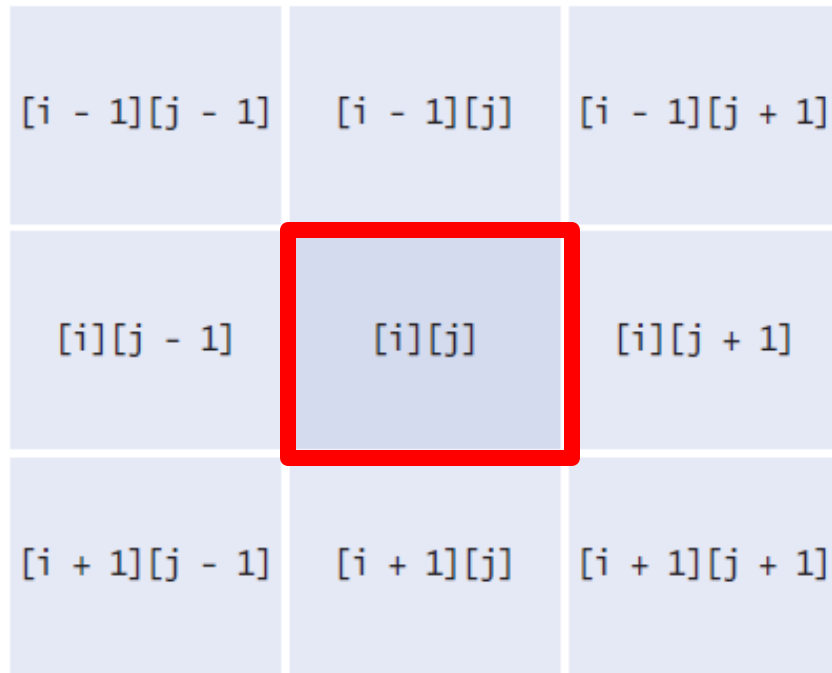


```
for i in range(COUNTRIES):  
    # Process the i-th row  
    for j in range(MEDALS) :  
        # Process the j-th column in the i-th row  
        print("%8d" % counts[i][j], end="")  
    print() # new line at the end of each row
```

```
0      3      0  
0      0      1  
0      0      1  
1      0      0  
0      0      1  
3      1      1  
0      1      0  
1      0      1
```

Locating Neighboring Elements

- Some programs that work with two-dimensional lists (aka tables) need to **locate the elements that are adjacent to an element**
- This task is particularly common in games
- You are at location i, j
- **Watch out for edges!**
 - Don't go off the 'board'



Printing a Single Row or Column

✦ Print the **i**-th row (RxC table)

```
for j in range(C):  
    print(C[i][j])
```

✦ Print the **j**-th column (RxC table)

```
for i in range(R):  
    print(C[i][j])
```

✦ Since each row is a list, in this case we can also simply write:

```
print(C[i])
```

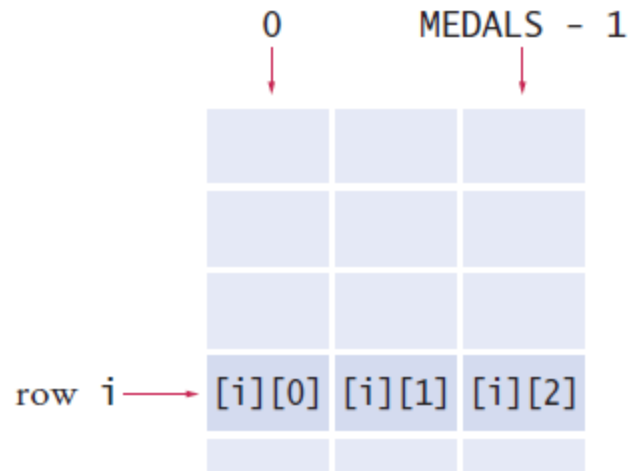
This will print: `C[i][0], C[i][1],...`

...but with the default list printing format

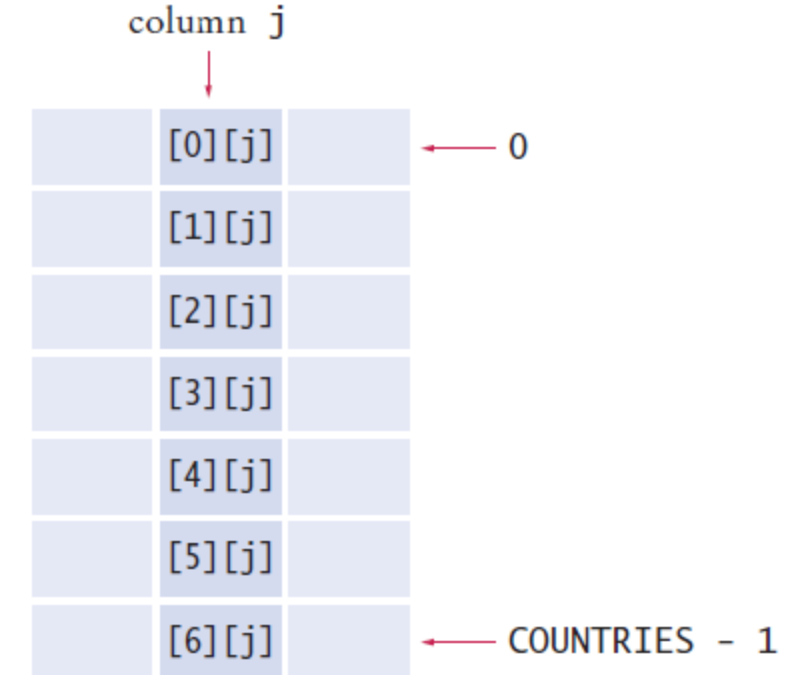
Adding Rows and Columns

Sum of the i-th row

```
i = 3 # the row we're interested in summing
rowtotal = 0
for j in range(MEDALS):
    rowtotal = rowtotal + counts[i][j]
```



Sum of the j-th column



```
j = 1 # the col we're interested in summing
coltotal = 0
for i in range(COUNTRIES):
    coltotal = coltotal + counts[i][j]
```

Using Tables With Functions

- Passing a table to a function is **not different** from passing a simple list
- Inside the function, you will want to recover the dimensions of the table to be able to “scan it”. If `values` is a table, then:
 - `len(values)` is the number of rows
 - `len(values[i])` is the **number of columns in the i-th row**
- Example: function to sum all elements in a table:

```
def sum(values) :  
    total = 0  
    for i in range(len(values)) :  
        for j in range(len(values[i])) :  
            total = total + values[i][j]  
    return total
```

Actually, each row can have a different number of columns (elements of the inner list).
But please avoid that!!

Exercise [P6.35 – Slightly Modified]

- You are given a table (`heights`) whose values correspond to the height of a terrain at different points in a square.
- Write a function `floodMap(heights, waterLevel)` that prints out a flood map, showing which of the points in the terrain would be flooded if the water level was the given value. In the flood map, **print a ~ for each flooded point and a space for each point that is not flooded.**
- Then write a program that shows how the terrain gets flooded when the water level increases **in ten steps from the lowest point in the terrain to the highest.**

Esercizio [P6.35]

★ Example: if the map is:

3	3	4
6	3	5
2	4	1

with **waterLevel = 3** the function will print:

~	~	
	~	
~		~

with **waterLevel = 5** the function will print:

~	~	~
	~	~
~	~	~

Summary

Summary: Lists

- A list is a container that stores a sequence of values
- Each individual element in a list is accessed by an integer index `i`, using the notation `list[i]`
- A list index must be at least zero and less than the number of elements in the list
- An out-of-range error, which occurs if you supply an invalid list index, can cause your program to terminate
- You can iterate over the `index` values or the `elements` of a list

Summary: Lists

- A list **reference** specifies the location of a list. Copying the reference yields a second reference to the **same** list
- A **linear search** inspects elements in sequence until a match is found
- Use a **temporary** variable when **swapping** elements
- Lists can occur as **function parameters** and **return values**

Summary: Lists

- When calling a function with a list argument, the function receives a list **reference**, not a copy of the list
- A tuple is created as a comma-separated sequence enclosed in parentheses
- By combining fundamental algorithms, you can solve complex programming tasks
- You should be familiar with the implementation of fundamental algorithms so that you can adapt them

Summary: Lists

- Use a two-dimensional list to store tabular data
- Individual elements in a two-dimensional list are accessed by using two index values, `table[i][j]`

Built-In Operations For Lists

- Use the `.insert()` method to insert a new element at any position in a list
- The `in` operator tests whether an element is contained in a list
- Use the `.pop()` method to remove an element from any position in a list
- Use the `.remove()` method to remove an element from a list by value
- Two lists can be concatenated using the plus (+) operator
- Use the `list()` function to copy lists

Built-In Operations For Lists

- Use the slice operator (`:`) to extract a sublist or substrings

Thanks

- Part of these slides are [edited versions] of those originally made by **Prof Giovanni Squillero** (Teacher of Course 1)
- Part of these slides are [edited versions] of those originally made by **Prof Massimo Poncino** (Teacher of one of the courses in Italian)

License

- These slides are distributed under the Creative Commons license “Attribution - Noncommercial - ShareAlike 2.5 (CC BY-NC-SA 2.5)”
- You are free:
 - to reproduce, distribute, communicate to the public, exhibit in public, represent, perform and recite this work
 - to modify this work
- Under the following conditions:
 - **Attribution** — You must attribute the work to the original authors, and you must do so in a way that does not suggest that they endorse you or your use of the work.
 - **Non-Commercial** — You may not use this work for commercial purposes.
 - **Share Alike** — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same or equivalent license as this one.
- <http://creativecommons.org/licenses/by-nc-sa/2.5/it/>

